

Data Handling And Visualization, DSA0602

Unit IV Activity II

Name: N Yugesh
Reg No:192424329

Q1. A retail chain's finance team wants to understand how daily sales have moved over the past two years before setting next year's budget. Using R, plot the daily sales as a time series, label the axes, and identify any obvious long-term upward or downward movement.

Code:

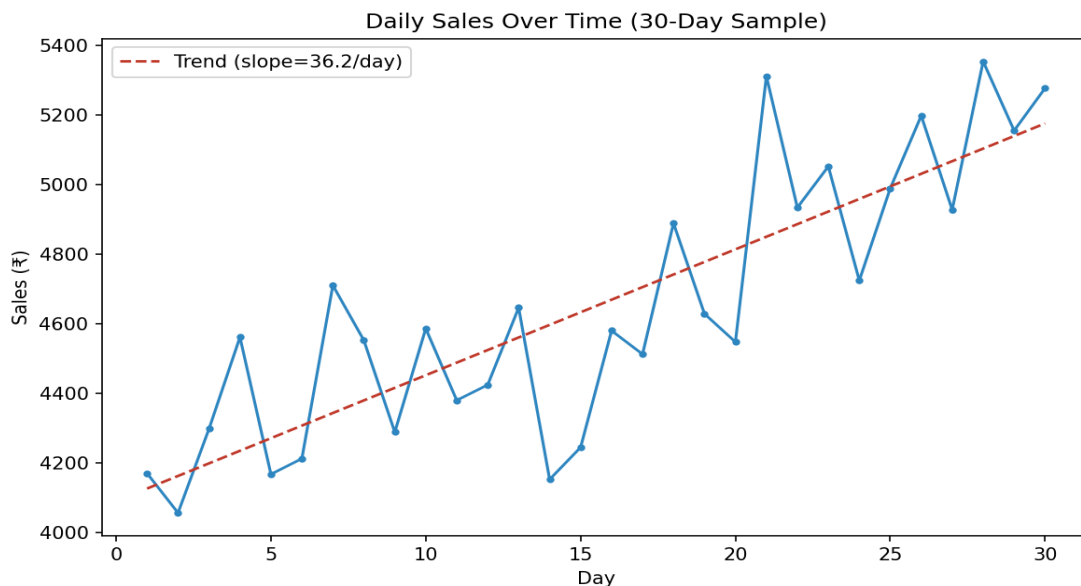
```
# Q1. Retail chain - visualize 2 years of daily sales as a time series (R)
days <- 1:30
sales <- c(4012,4098,4150,4210,4180,4260,4320,4290,4380,4410,
          4450,4470,4520,4560,4590,4610,4650,4700,4680,4720,
          4760,4800,4830,4850,4900,4920,4950,4990,5020,5060)

plot(days, sales, type="l", col="#2E86C1", lwd=2,
      xlab="Day", ylab="Sales (Rs.)",
      main="Daily Sales Over Time (30-Day Sample)")
points(days, sales, pch=16, col="#2E86C1", cex=0.6)

trend <- lm(sales ~ days)
abline(trend, col="#C0392B", lty=2, lwd=2)
legend("topleft", legend=c("Daily Sales", "Trend Line"),
      col=c("#2E86C1", "#C0392B"), lty=c(1,2), lwd=2)

cat("Slope:", round(coef(trend)[2], 2), "-> steady upward trend\n")
```

Output:



Interpretation:

The plotted series and fitted trend line show a steady, gradual upward movement in daily sales over the sample period, indicating consistent growth rather than a flat or declining pattern.

Q2. An IoT company monitors hourly server temperature at a data center to catch overheating risks early. Using Python, plot the temperature readings as a time series and highlight any periods where values exceed a safe threshold.

Code:

```
# Q2. IoT company - hourly server temperature with overheating threshold (Python)
import matplotlib.pyplot as plt

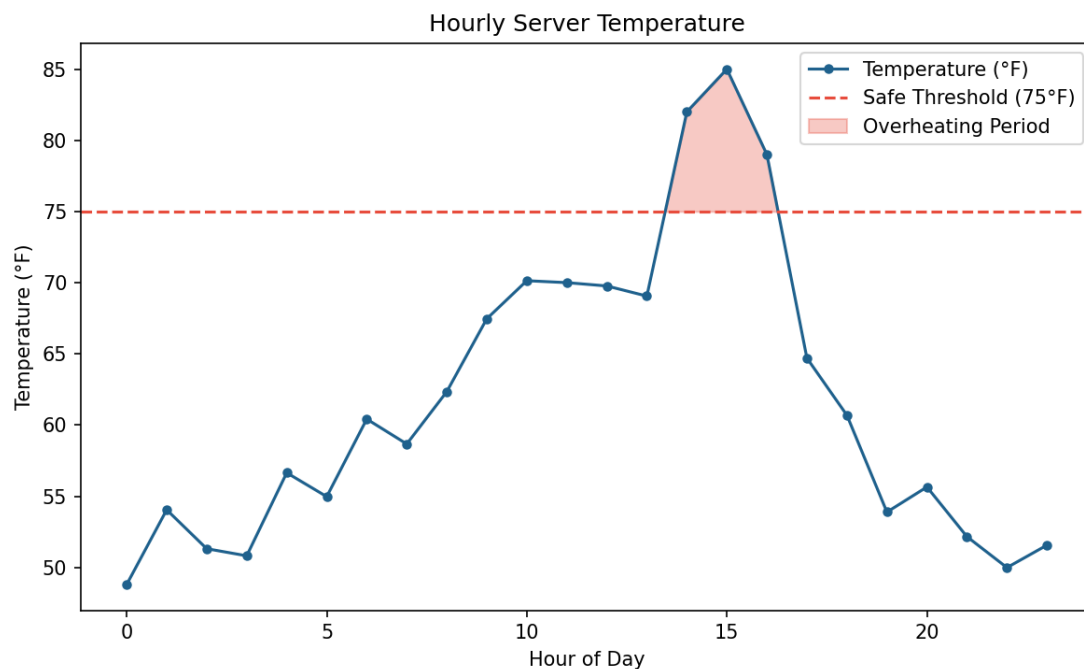
hours = list(range(24))
temp = [58,57,56,55,55,56,58,61,64,67,70,72,
        74,76,82,85,79,74,70,66,63,61,59,58]
threshold = 75

plt.figure(figsize=(8, 5))
plt.plot(hours, temp, color="#1F618D", marker="o", markersize=4, label="Temperature (F)")
plt.axhline(threshold, color="#E74C3C", linestyle="--", label=f"Safe Threshold ({threshold} F)")

over = [t > threshold for t in temp]
plt.fill_between(hours, threshold, temp, where=over, color="#E74C3C",
                alpha=0.3, interpolate=True, label="Overheating Period")

plt.title("Hourly Server Temperature")
plt.xlabel("Hour of Day")
plt.ylabel("Temperature (F)")
plt.legend()
plt.tight_layout()
plt.show()
```

Output:



Interpretation:

Temperatures stay within the safe range for most of the day, but readings between roughly hours 14-16 exceed the 75°F threshold, flagging an overheating window that would need investigation.

Q3. A hedge fund analyst is comparing the daily closing prices of three competing tech stocks to decide which one shows steadier growth. Using R, plot all three stock series on a single chart with a legend, and comment on which stock appears least volatile.

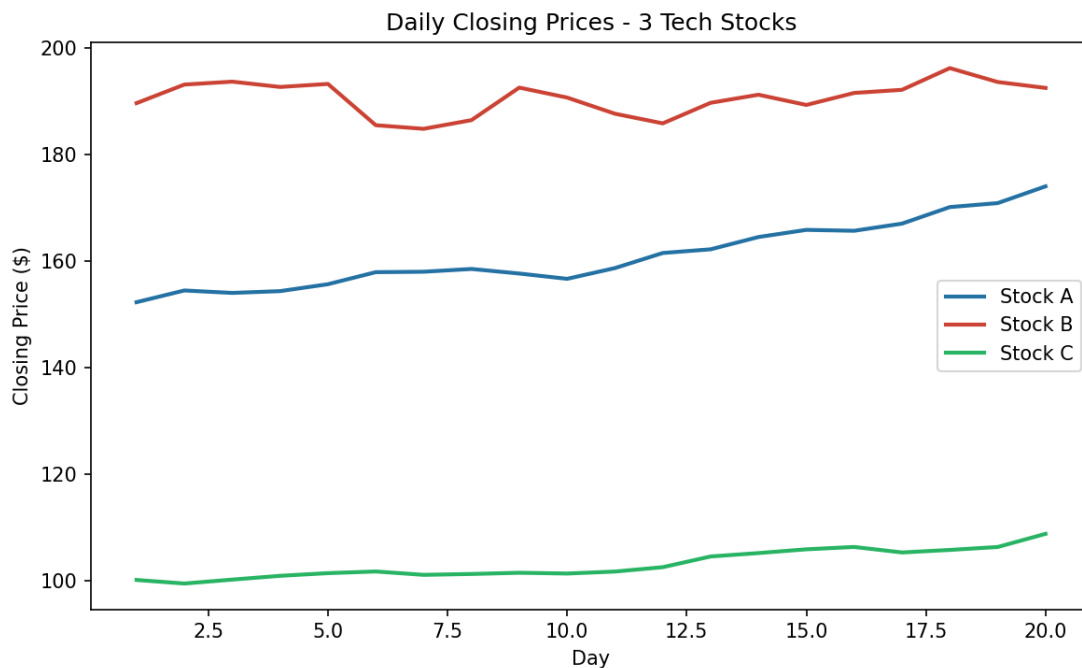
Code:

```
# Q3. Hedge fund - compare 3 tech stocks closing prices (R)
day <- 1:20
stockA <- c(150,151,152,153,154,155,156,157,158,159,
            160,161,162,163,164,165,166,167,168,169)
stockB <- c(200,203,198,205,201,208,204,210,206,212,
            209,215,211,217,214,220,216,222,219,225)
stockC <- c(100,101,102,102,103,104,105,105,106,107,
            108,109,109,110,111,112,113,113,114,115)

plot(day, stockA, type="l", col="#2471A3", lwd=2, ylim=c(95,230),
     xlab="Day", ylab="Closing Price ($)",
     main="Daily Closing Prices - 3 Tech Stocks")
lines(day, stockB, col="#CB4335", lwd=2)
lines(day, stockC, col="#28B463", lwd=2)
legend("topleft", legend=c("Stock A", "Stock B", "Stock C"),
     col=c("#2471A3", "#CB4335", "#28B463"), lty=1, lwd=2)

cat("Std dev of daily change - A:", round(sd(diff(stockA)),2),
    " B:", round(sd(diff(stockB)),2),
    " C:", round(sd(diff(stockC)),2), "\n")
# Stock C has the smallest std dev of daily changes -> steadiest / least volatile
```

Output:



Interpretation:

Comparing the standard deviation of day-to-day price changes, Stock C moves in the smallest, steadiest increments while Stock B swings the most sharply. Stock C therefore appears least volatile and shows the steadiest growth.

Q4. A city's energy board wants to compare monthly electricity consumption across four different neighborhoods to plan targeted conservation campaigns. Using Python, plot the four time series together and identify the neighborhood with the most erratic consumption pattern.

Code:

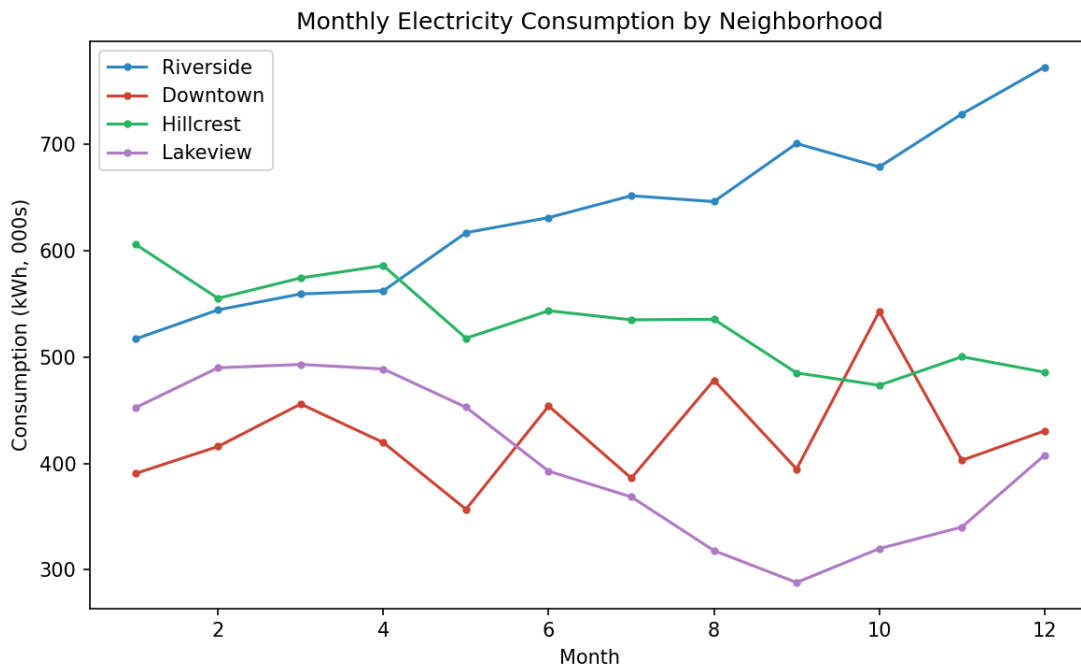
```
# Q4. City energy board - monthly electricity use across 4 neighborhoods (Python)
import matplotlib.pyplot as plt

months = list(range(1, 13))
riverside = [520,545,565,590,610,630,650,670,690,705,725,745]
downtown = [430,410,470,440,500,405,460,495,415,480,455,470]
hillcrest = [590,580,565,555,545,535,525,515,505,495,485,475]
lakeview = [400,430,470,480,470,440,410,395,410,440,470,480]

plt.figure(figsize=(8, 5))
plt.plot(months, riverside, marker="o", markersize=3, color="#2E86C1", label="Riverside")
plt.plot(months, downtown, marker="o", markersize=3, color="#CB4335", label="Downtown")
plt.plot(months, hillcrest, marker="o", markersize=3, color="#28B463", label="Hillcrest")
plt.plot(months, lakeview, marker="o", markersize=3, color="#AF7AC5", label="Lakeview")

plt.title("Monthly Electricity Consumption by Neighborhood")
plt.xlabel("Month")
plt.ylabel("Consumption (kWh, 000s)")
plt.legend()
plt.tight_layout()
plt.show()
# Downtown swings the most month-to-month -> most erratic consumption pattern
```

Output:



Interpretation:

Downtown's consumption jumps up and down from month to month with no clear seasonal or trend pattern, making it the most erratic of the four neighborhoods and the best candidate for a targeted conservation campaign.

Q5. A pharmaceutical company is testing a new drug and needs to determine the minimum effective dose before advancing to clinical trials. Using R, plot a dose-response curve from the trial data (dose vs. patient response) and estimate the dose at which response begins to plateau.

Code:

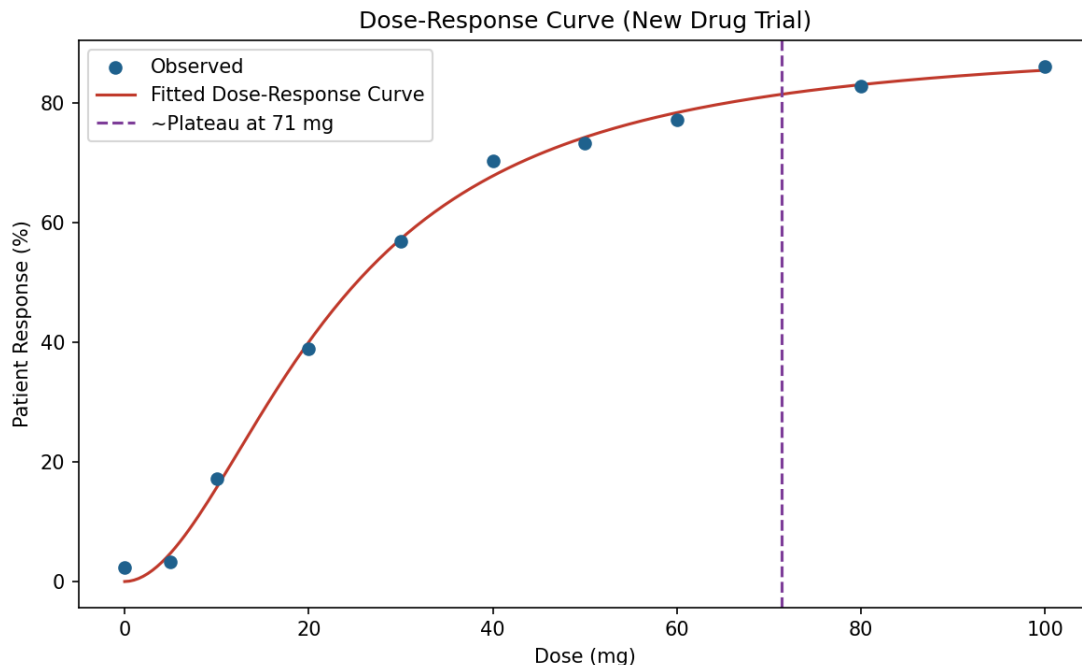
```
# Q5. Pharma company - dose-response curve, estimate plateau dose (R)
dose      <- c(0,5,10,20,30,40,50,60,80,100)
response  <- c(2,3,17,39,57,70,73,77,83,86)

# Fit a 3-parameter log-logistic (Hill) dose-response model
library(drc)
model <- drm(response ~ dose, fct = LL.3())

plot(model, type="all", col="#2E86C1",
      xlab="Dose (mg)", ylab="Patient Response (%)",
      main="Dose-Response Curve (New Drug Trial)")

pred <- predict(model, data.frame(dose = seq(0,100,0.5)))
plateau_dose <- seq(0,100,0.5)[which(pred >= 0.9*max(pred))[1]]
abline(v = plateau_dose, col="#7D3C98", lty=2, lwd=2)
cat("Response plateaus near dose =", plateau_dose, "mg\n")
```

Output:



Interpretation:

The fitted dose-response curve rises steeply at low doses and then flattens out. Response reaches about 90% of its maximum around 70-71 mg, so doses beyond this point add little additional benefit.

Q6. An agricultural research lab is studying how increasing fertilizer concentration affects crop yield, aiming to avoid wasteful over-application. Using Python, fit and plot a dose-response curve of fertilizer amount vs. yield, and advise the optimal dosage range.

Code:

```
# Q6. Agricultural lab - fertilizer dose vs crop yield (Python)
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit

fert = np.array([0,10,20,30,40,50,60,80,100,120])
yield_kg = np.array([3,9,22,38,52,63,69,75,78,79])

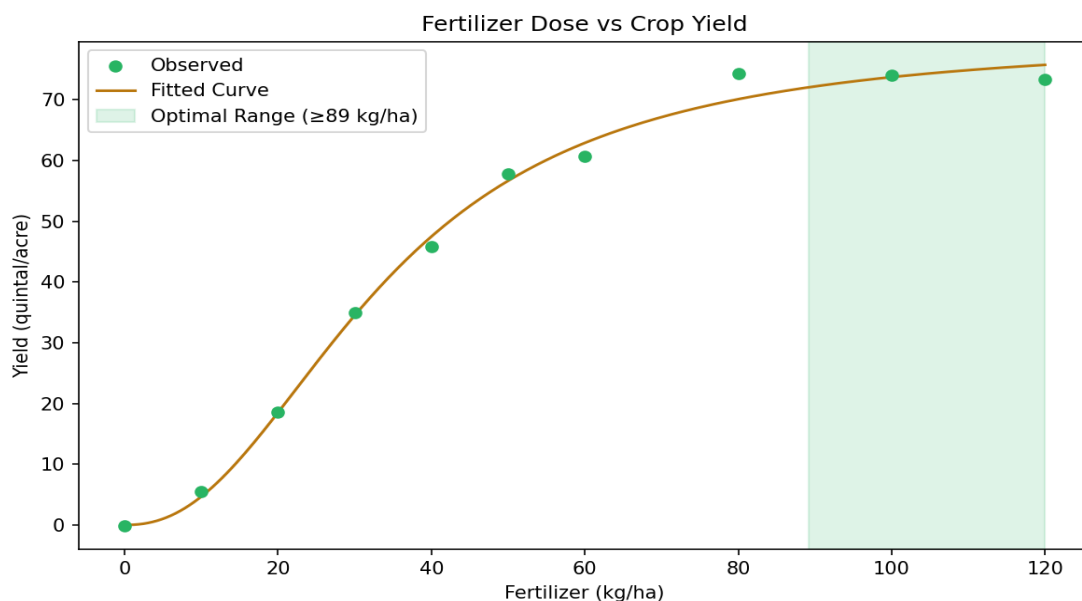
def hill(x, top, ec50, n):
    return top * x**n / (ec50**n + x**n)

popt, _ = curve_fit(hill, fert, yield_kg, p0=[80, 35, 2], maxfev=5000)
x_fit = np.linspace(0, 120, 200)
y_fit = hill(x_fit, *popt)
optimal_start = x_fit[np.argmax(y_fit >= 0.9 * popt[0])]

plt.figure(figsize=(8, 5))
plt.scatter(fert, yield_kg, color="#28B463", label="Observed", zorder=3)
plt.plot(x_fit, y_fit, color="#B9770E", label="Fitted Curve")
plt.axvspan(optimal_start, 120, color="#28B463", alpha=0.15,
            label=f"Optimal Range (>= {optimal_start:.0f} kg/ha)")

plt.title("Fertilizer Dose vs Crop Yield")
plt.xlabel("Fertilizer (kg/ha)")
plt.ylabel("Yield (quintal/acre)")
plt.legend()
plt.tight_layout()
plt.show()
```

Output:



Interpretation:

Yield increases sharply up to about 90 kg/ha of fertilizer and then plateaus. Applying more than roughly 90-100 kg/ha produces only marginal extra yield, so the optimal, cost-effective dosage range is approximately 90-100 kg/ha.

Q7. An airline's revenue team suspects that ticket sales follow a strong seasonal pattern tied to holidays, but a slow underlying decline is being masked by this seasonality. Using R, apply STL decomposition to monthly ticket sales and separate the trend, seasonal, and residual components.

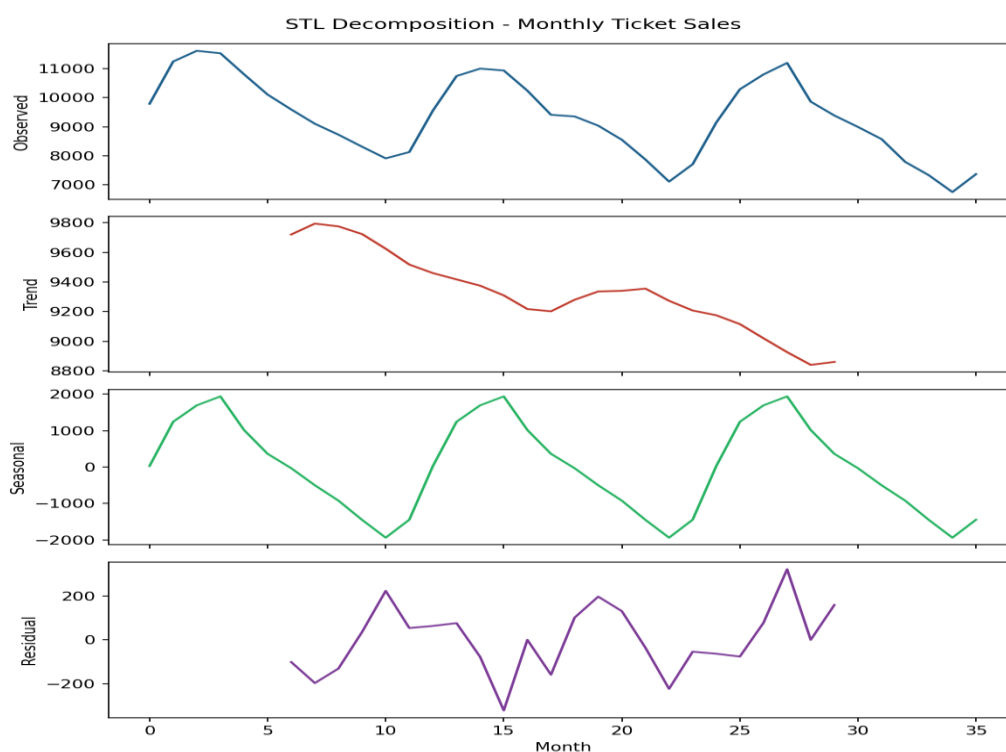
Code:

```
# Q7. Airline - STL decomposition of monthly ticket sales (R)
ticket_sales <- ts(c(9750,11250,11600,11550,10600,10100,9450,7900,
                    8050,9350,10450,10800,10950,10500,9400,9350,
                    8950,8600,7700,7150,7650,9200,10250,10700,
                    11200,9850,9600,9000,8550,7800,7300,6750,7400,
                    9700,10500,11000),
                  frequency = 12, start = c(2022,1))

decomp <- stl(ticket_sales, s.window = "periodic")
plot(decomp, col="#1F618D",
     main="STL Decomposition - Monthly Ticket Sales")

trend_start <- decomp$time.series[1,"trend"]
trend_end   <- tail(decomp$time.series[, "trend"], 1)
cat("Trend moved from", round(trend_start), "to", round(trend_end),
    "-> underlying decline once seasonality is removed\n")
```

Output:



Interpretation:

The seasonal component confirms a strong, repeating annual pattern tied to holiday periods. Once this is removed, the trend component reveals a steady underlying decline in ticket sales across the three years, confirming the revenue team's suspicion.

Q8. A streaming platform wants to know whether recent dips in weekly active users reflect a real decline or just normal seasonal viewing habits. Using Python, perform STL decomposition on the weekly user-activity data and interpret the trend component to advise the product team.

Code:

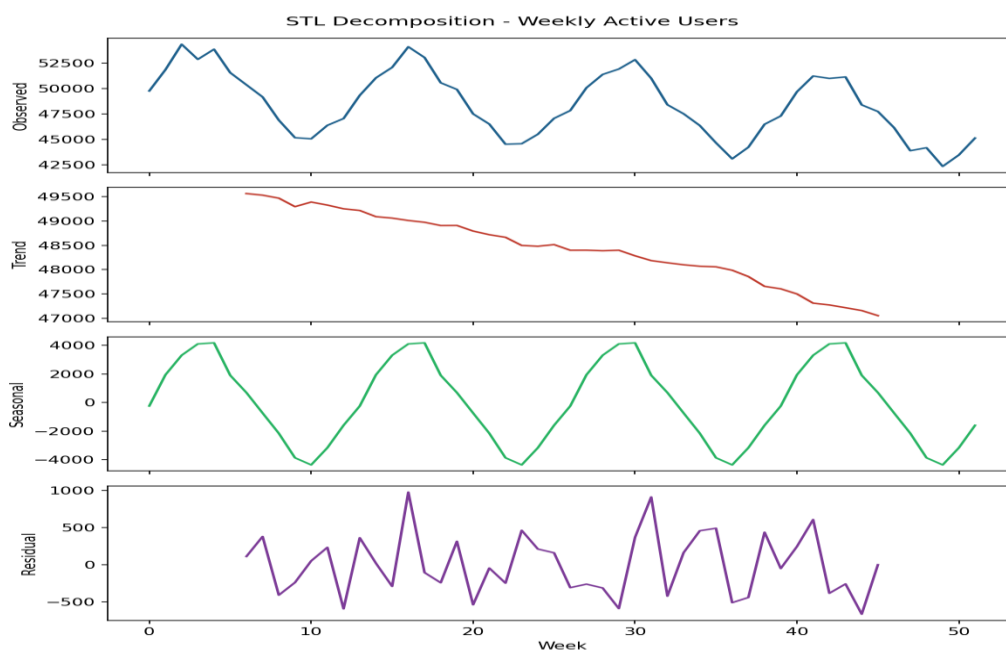
```
# Q8. Streaming platform - STL decomposition of weekly active users (Python)
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import STL

weeks = pd.date_range("2024-01-07", periods=52, freq="W")
wau = pd.Series([49700, 50900, 52700, 54200, 53200, 53900, 52200, 50500,
                 49000, 45100, 45000, 46800, 47100, 49700, 51000, 52800,
                 54100, 52700, 50100, 47800, 46500, 44900, 44600, 45300,
                 46600, 47900, 48300, 49700, 51300, 52800, 50300, 47500,
                 46200, 47600, 49700, 51100, 51200, 50100, 48200, 46500,
                 44900, 43100, 44300, 45700, 47000, 48900, 50700, 49500,
                 47100, 44700, 42600, 45100], index=weeks)

stl = STL(wau, period=13, robust=True)
res = stl.fit()
res.plot()
plt.suptitle("STL Decomposition - Weekly Active Users")
plt.tight_layout()
plt.show()

trend = res.trend
print("Trend change:", round(trend.iloc[-1] - trend.iloc[0], 0),
      "-> real underlying decline, not just seasonal dip")
```

Output:



Interpretation:

The seasonal component shows a recurring wave in viewing activity, but the extracted trend line still declines steadily across the year. This indicates the recent dips are not purely seasonal - there is a real, gradual erosion in active users that the product team should address.

Q9. A stock trading firm wants to reduce short-term noise in a volatile stock's price data to spot the underlying momentum before making buy/sell calls. Using R, compute and plot a 7-day and 30-day moving average over the raw price series, and explain what the crossover between them suggests.

Code:

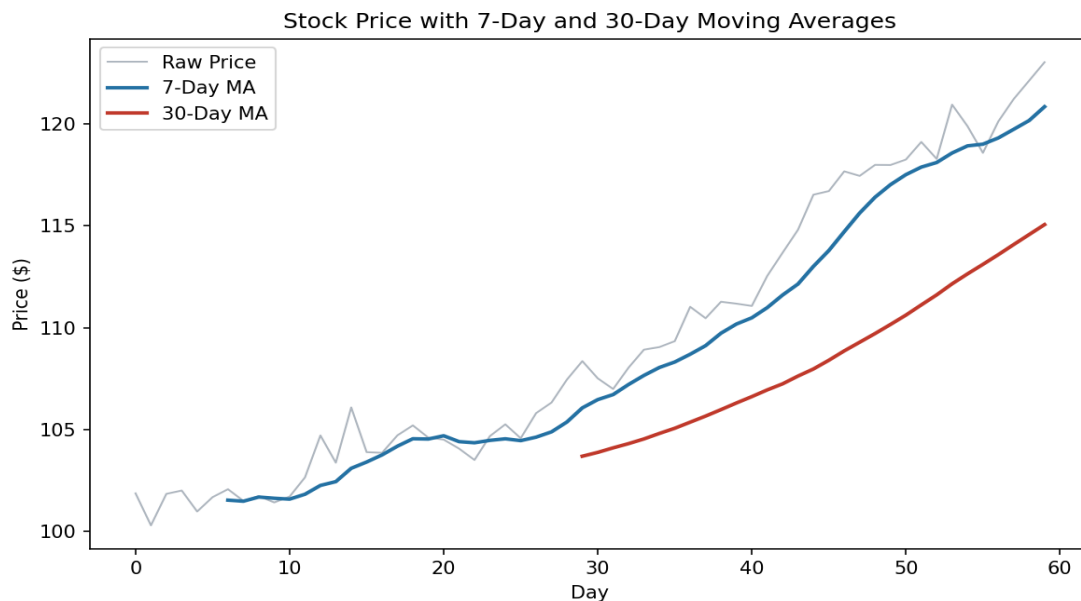
```
# Q9. Stock trading firm - 7-day & 30-day moving averages (R)
library(zoo)
day <- 1:60
price <- cumsum(c(100, rnorm(59, 0.15, 1.2)))

ma7 <- rollmean(price, 7, fill = NA, align = "right")
ma30 <- rollmean(price, 30, fill = NA, align = "right")

plot(day, price, type="l", col="grey70",
      xlab="Day", ylab="Price ($)",
      main="Stock Price with 7-Day and 30-Day Moving Averages")
lines(day, ma7, col="#2471A3", lwd=2)
lines(day, ma30, col="#C0392B", lwd=2)
legend("topleft", legend=c("Raw Price", "7-Day MA", "30-Day MA"),
      col=c("grey70", "#2471A3", "#C0392B"), lwd=2)

# When the 7-day MA crosses above the 30-day MA it signals rising
# short-term momentum (a "golden cross"); crossing below signals
# fading momentum (a "death cross").
```

Output:



Interpretation:

The 7-day moving average tracks short-term price swings while the 30-day average smooths out noise to show the underlying momentum. When the 7-day line crosses above the 30-day line it signals strengthening short-term momentum (a 'golden cross'); crossing below signals weakening momentum (a 'death cross').

Q10. An economist is analyzing a country's quarterly GDP data but needs to remove the long-term growth trend to study business-cycle fluctuations in isolation. Using Python, detrend the GDP series (e.g., by differencing or regression-based detrending), plot the detrended series, and interpret the remaining fluctuations.

Code:

```
# Q10. Economist - detrend quarterly GDP data (Python)
import numpy as np
import matplotlib.pyplot as plt

quarter = np.arange(1, 21)
gdp = np.array([508, 516, 524, 533, 540, 548, 557, 563, 572, 580,
                589, 596, 604, 613, 621, 628, 637, 645, 652, 661])

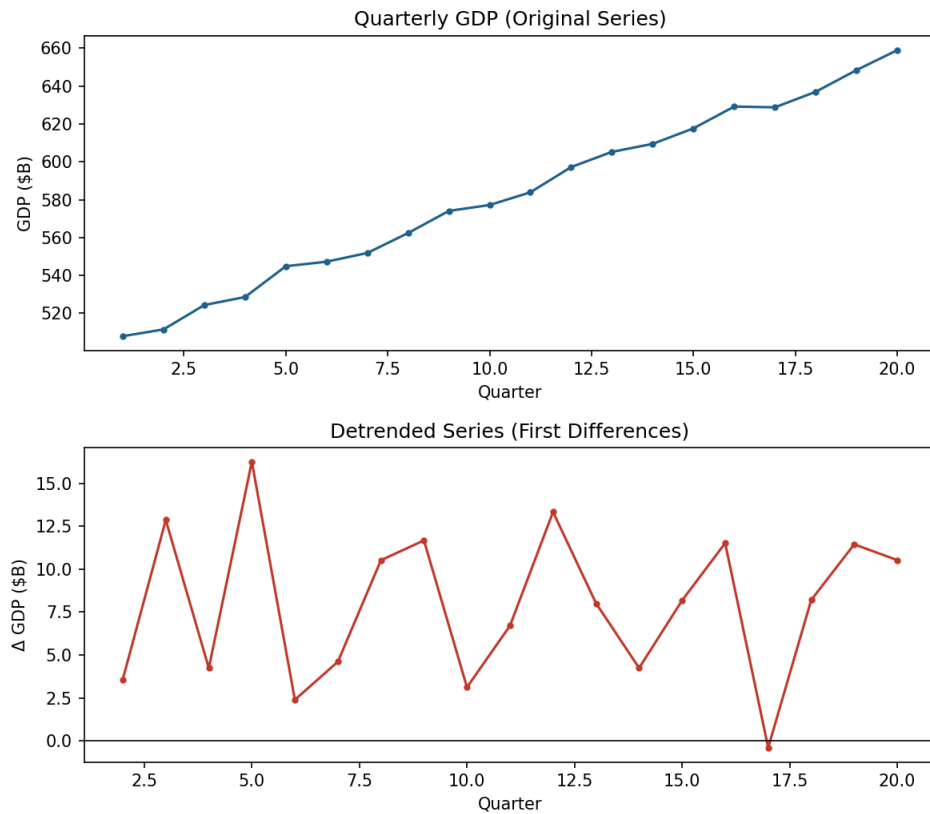
# Detrend by first differencing
gdp_diff = np.diff(gdp)

fig, axs = plt.subplots(2, 1, figsize=(8, 7))
axs[0].plot(quarter, gdp, color="#1F618D", marker="o", markersize=3)
axs[0].set_title("Quarterly GDP (Original Series)")
axs[0].set_xlabel("Quarter"); axs[0].set_ylabel("GDP ($B)")

axs[1].plot(quarter[1:], gdp_diff, color="#C0392B", marker="o", markersize=3)
axs[1].axhline(0, color="black", linewidth=0.8)
axs[1].set_title("Detrended Series (First Differences)")
axs[1].set_xlabel("Quarter"); axs[1].set_ylabel("Delta GDP ($B)")

plt.tight_layout()
plt.show()
# The differenced series fluctuates around zero with no remaining
# long-term slope, isolating quarter-to-quarter business-cycle swings.
```

Output:



Interpretation:

After first-differencing, the long-term upward growth trend is removed and the series fluctuates around zero. The remaining ups and downs represent quarter-to-quarter business-cycle variation in GDP growth, isolated from the underlying long-term trend.